

TP2 – Python

Pr. Houda BENHAR

Exercice 1.

```
from math import pi
def cube(x):
    return x**3
def volume_sphere(r):
    return 4*pi*cube(r)/3
try:
    rayon = float(input("Saisir la valeur du rayon: "))
except:
    print("Valeur invalide")
    exit()
print("le volume de votre sphere est: ", volume_sphere(rayon))
```

Exercice 2.

```
# Module math_operations.py

def addition(a, b):
    return a + b

def soustraction(a, b):
    return a - b

def multiplication(a, b):
    return a * b

def division(a, b):
    if b != 0:
        return a / b
    else:
        return "Erreur: division par zéro"

# Programme utilisant le module math_operations.py

import math_operations

# Utilisation des fonctions du module
resultat_addition = math_operations.addition(5, 3)
print("Résultat de l'addition:", resultat_addition)

resultat_soustraction = math_operations.soustraction(10, 4)
print("Résultat de la soustraction:", resultat_soustraction)

resultat_multiplication = math_operations.multiplication(7, 2)
print("Résultat de la multiplication:", resultat_multiplication)
```

```
resultat_division = math_operations.division(9, 3)
print("Résultat de la division:", resultat_division)
```

Exercice 3.

```
def longueur_chaine(chaine):
    longueur = 0
    for caractere in chaine:
        longueur += 1
    return longueur

print(longueur_chaine("Bonjour")) # Résultat : 7
```

Exercice 4.

```
def compter_voyelles(chaine):
    voyelles = "aeiouAEIOU"
    compteur = 0
    for char in chaine:
        if char in voyelles:
            compteur += 1
    return compteur

print(compter_voyelles("hello")) # Résultat : 2
```

Ou

```
def compter_voyelles(chaine):
    voyelles = "aeiou"
    compteur = 0
    for char in chaine:
        if char.lower() in voyelles:
            compteur += 1
    return compteur

print(compter_voyelles("hello")) # Résultat : 2
```

Exercice 5.

```
def supprimer_espaces(chaine):
    return chaine.replace(" ", "")

print(supprimer_espaces("Bonjour le monde")) # Résultat :
"Bonjourlemonde"
```

Exercice 6.

Le plus simple consiste à dire que le nombre de mots de la phrase correspond au nombre d'espaces plus 1. Voici une correction possible:

```
# La phrase à traiter (vous pourriez aussi la lire via un appel à input).
sentence = "python est un langage merveilleux"
```

```

# Calcul du nombre d'espaces
total = sentence.count(" ")
# Le nombre de mots correspond au nombre d'espaces + 1
total += 1
if total == 1:
    print('La phrase "', sentence, '" contient 1 mot')
else:
    print('La phrase "', sentence, '" contient', total, 'mots')

# Ou

def compter_mots(phrase):
    mots = phrase.split()
    nombre_mots = len(mots)
    return nombre_mots

```

Exercice 7.

Pour répondre à cet exercice, le plus simple serait de déterminer la position du premier espace dans la chaîne. Une fois cette position déterminée, il ne restera plus qu'à extraire la chaîne et la capitaliser. Voici une correction possible:

```

sentence = "python est un langage merveilleux"

# Calcul de la position du premier espace
pos = sentence.find(" ")
if pos == -1:
    # Dans ce cas, la variable sentence ne contient qu'un seul mot.
    word = sentence.capitalize()
    print(word)
else:
    # On extrait le premier mot (rappel : l'indice de fin est exclusif).
    word = sentence[0:pos]
    word = word.capitalize()
    print(word)

```

Exercice 8.

```

data = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
atpos = data.find('@')
spos = data.find(' ', atpos)
host = data[atpos+1:spos]
print(host)

```

Exercice 9.

```

def estdistinct(n):
    distinct = True
    if n > 0:
        ch = str(n)
        for lettre in ch:
            if ch.count(lettre) > 1:
                distinct = False

```

```
        break
    if etat == True:
        print(n, ' est distinct')
    else:
        print(n, ' est non distinct')
else:
    print('le nombre est négatif ')
print(estdistinct(12344))
```